



Estudo abstrato de “Classes” em Programação Orientada a Objetos

NOMES	RA
João Paulo Nunes Andrade	25002703
Caique Andraus	25006881
Gustavo Antoniazi	25007671
Pedro Carvalho	21005320
Arthur Carvalho	25015941

PROFESSOR	EMAIL
Leonardo Rezende Costa	leonardo.costa@puc-campinas.edu.br

1. O que usamos?

Para modelar a entidade **Contas Bancárias**, definimos um conjunto equilibrado de atributos (informações) e métodos (operações/ações) para simular o comportamento essencial de uma conta corrente com limite de crédito.

Atributos (Dados/Estado):

- **numeroConta** (*String*): Identificador único da conta.
- **titular** (*String*): Nome do proprietário da conta.
- **saldo** (*double*): Valor monetário atualmente disponível na conta.
- **limiteEspecial** (*double*): Crédito pré-aprovado que permite saques mesmo quando o saldo for zero.

Métodos (Comportamento/Ações):

- **ContaBancaria(...)** (Construtor): Inicializa a conta com os dados do titular e define o saldo inicial como zero.
- **depositar(double valor)**: Realiza a inclusão de valores na conta.
- **sacar(double valor)**: Realiza a retirada de valores, validando se há saldo + limite suficiente.
- **transferir(double valor, ContaBancaria destino)**: Transfere dinheiro para outra instância de **ContaBancaria**.
- **exibirExtrato()**: Imprime o resumo atual das informações financeiras.

2. Por que usamos?

A escolha do objeto **Conta Bancária** atende perfeitamente ao pedido por ser um conceito **intuitivo e cotidiano**, mas que exige **regras de negócio claras e proteção de dados** (complexidade na medida certa).

- **Encapsulamento (Proteção dos dados)**: Atributos financeiros não devem ser alterados diretamente por qualquer parte do código (por exemplo, ninguém pode fazer `conta.saldo = 1000000;`). Usamos o modificador **private** nos atributos para garantir que as alterações ocorram apenas por meio dos métodos da própria

classe (**depositar** ou **sacar**), validando se a operação é permitida.

- **Garantia de Regra de Negócio (Consistência):** O método **sacar** valida se a conta possui fundos suficientes (saldo + limite) antes de debitar. Se a conta não tiver fundos suficientes, a operação é negada de forma segura, mantendo os dados coerentes.
- **Reutilização e Modularidade:** A classe serve como um modelo (molde). A partir desse único modelo, podemos criar milhares de objetos/instâncias de contas bancárias diferentes na memória do computador.

4. Código

O código abaixo contém tanto a classe **ContaBancaria** quanto uma classe **Main** com um exemplo de uso prático para testar as operações.

Linguagem: Java

Arquivos: ContaBancaria.java, Main.java

Link::

ContaBancaria.java

```
public class ContaBancaria {  
  
    // 1. Atributos Privados (Encapsulamento)  
    private String numeroConta;  
    private String titular;  
    private double saldo;  
    private double limiteEspecial;  
  
    // 2. Construtor (Inicialização do objeto)  
    public ContaBancaria(String numeroConta, String titular, double limiteEspecial) {  
        this.numeroConta = numeroConta;  
        this.titular = titular;  
        this.limiteEspecial = limiteEspecial;  
        this.saldo = 0.0; // Toda conta nova inicia com saldo zerado  
    }  
  
    // 3. Métodos com Regras de Negócio  
  
    // Operação de Depósito  
    public void depositar(double valor) {  
        if (valor > 0) {  
            this.saldo += valor;  
            System.out.println("✅ Depósito de R$ " + valor + " realizado com sucesso para "
```

```

+ this.titular);
    } else {
        System.out.println("✗ Valor de depósito inválido!");
    }
}

// Operação de Saque com verificação de saldo + limite
public boolean sacar(double valor) {
    if (valor <= 0) {
        System.out.println("✗ O valor do saque deve ser maior que zero!");
        return false;
    }

    double saldoTotalDisponivel = this.saldo + this.limiteEspecial;

    if (valor <= saldoTotalDisponivel) {
        this.saldo -= valor;
        System.out.println("✓ Saque de R$ " + valor + " realizado com sucesso!");
        return true;
    } else {
        System.out.println("✗ Saque recusado para " + this.titular + ": Saldo e limite
insuficientes!");
        return false;
    }
}

// Operação de Transferência entre duas contas
public boolean transferir(double valor, ContaBancaria destino) {
    System.out.println("\n--- Iniciando Transferência ---");
    if (this.sacar(valor)) {
        destino.depositar(valor);
        System.out.println("✓ Transferência de R$ " + valor + " realizada para " +
destino.getTitular());
        return true;
    } else {
        System.out.println("✗ Transferência não realizada.");
        return false;
    }
}

// Impressão do estado da conta
public void exibirExtrato() {
    System.out.println("\n-----");
    System.out.println("    EXTRATO BANCÁRIO    ");
    System.out.println("-----");
    System.out.println("Titular: " + this.titular);
    System.out.println("Conta: " + this.numeroConta);
    System.out.println("Saldo Atual: R$ " + this.saldo);
    System.out.println("Limite Especial: R$ " + this.limiteEspecial);
    System.out.println("Saldo Total Disponível: R$ " + (this.saldo + this.limiteEspecial));
    System.out.println("-----");
}

```

```
// 4. Getters e Setters (Acesso controlado)
public String getNumeroConta() {
    return numeroConta;
}

public String getTitular() {
    return titular;
}

public double getSaldo() {
    return saldo;
}

public double getLimiteEspecial() {
    return limiteEspecial;
}

public void setLimiteEspecial(double limiteEspecial) {
    this.limiteEspecial = limiteEspecial;
}
}
```

Main.java

```
public class Main {
    public static void main(String[] args) {
        // Criando dois objetos (instâncias da classe ContaBancaria)
        ContaBancaria conta1 = new ContaBancaria("1001-X", "João Silva", 500.0);
        ContaBancaria conta2 = new ContaBancaria("2002-Y", "Maria Souza", 200.0);

        // Extrato Inicial
        conta1.exibirExtrato();

        // Operação 1: Depósito
        conta1.depositar(1000.0);

        // Operação 2: Saque dentro do saldo
        conta1.sacar(300.0);

        // Operação 3: Saque usando parte do limite especial
        conta1.sacar(900.0); // Saldo era 700. Usou 200 do limite especial (saldo fica -200)

        // Operação 4: Tentativa de saque estourando saldo + limite
        conta1.sacar(500.0); // Recusado

        // Operação 5: Transferência entre contas
        conta1.depositar(500.0); // Recupera saldo positivo
        conta1.transferir(200.0, conta2);
    }
}
```

```
// Exibir extratos finais
conta1.exibirExtrato();
conta2.exibirExtrato();
    }
}
```